

A Novel Perspective on the P vs NP Problem with an AI Co-Pilot Exploration Framework

Samuel Castillo

University of Massachusetts Amherst

April 4, 2026

- 1 Motivation
- 2 Formal Definitions
- 3 Feasibility and Runtime Growth
- 4 Intuition and NP-Completeness

Why This Problem Matters

Core idea

- The P vs. NP problem asks whether every efficiently verifiable problem is also efficiently solvable.
- It sits at the center of complexity theory, optimization, cryptography, and algorithm design.
- The practical issue is **computational feasibility**: which problems stay solvable under real limits on time, memory, energy, and hardware.
- This presentation reframes the paper around **Time-to-Solve (TTS)**, large-scale systems, and an **AI-assisted exploration framework**.

Real-World Motivation: Travel Systems and Optimization

- Large travel systems must forecast demand, allocate inventory, schedule under disruptions, and optimize itineraries.
- Many of these tasks resemble classical combinatorial optimization problems.
- The canonical example is the **Traveling Salesman Problem (TSP)**:

Given a set of locations and pairwise distances, what is the shortest route that visits each location once and returns to the origin?

- Exact solutions typically become infeasible at scale.
- Practical systems rely on heuristics, approximations, decomposition, and predictive models.

$$P = \left\{ L \subseteq \Sigma^* \mid \exists \text{ deterministic Turing machine } M, \exists k \in \mathbb{N}, \text{ such that } M \text{ decides } L \text{ in } O(n^k) \text{ time} \right\}$$

- Informally: problems solvable in polynomial time.
- Polynomial time is usually treated as the mathematical model of tractability.

$$\text{NP} = \left\{ L \subseteq \Sigma^* \mid \exists \text{ nondeterministic Turing machine } M, \exists k \in \mathbb{N}, \text{ such that } M \right.$$

Equivalent verifier-based definition:

$$x \in L \iff \exists y \in \Sigma^*, |y| \leq p(|x|), V(x, y) = 1$$

where V is a deterministic polynomial-time verifier.

Does $P = NP$?

Equivalent wording

Does every language that admits a polynomial-time verifier also admit a polynomial-time decision algorithm?

- If $P = NP$, many currently intractable search problems would become polynomial-time solvable.
- If $P \neq NP$, then the gap between verification and search is fundamental.

Time-to-Solve (TTS) and Feasibility

- Complexity is not just a theoretical label; it is a practical limit on resources.
- Feasibility depends on:
 - time,
 - memory,
 - energy and hardware,
 - scalability as input size grows.
- Polynomial growth is usually manageable.
- Exponential growth quickly outruns any realistic compute budget.

Scaling Behavior by Complexity Class

- $O(1)$: constant time
- $O(\log n)$: logarithmic time
- $O(n)$: linear time
- $O(n^2)$: quadratic time
- $O(2^n)$: exponential time

Key idea: the difference between polynomial and exponential time is not a small efficiency gap. It is often the difference between **practical** and **impossible**.

Real Runtime Comparison

Assume a machine performing approximately 10^9 basic operations per second.

Input n	$O(n)$	$O(n^2)$	$O(2^n)$
10	10^{-8} s	10^{-7} s	1.0×10^{-6} s
30	3×10^{-8} s	9×10^{-7} s	1.07 s
50	5×10^{-8} s	2.5×10^{-6} s	≈ 13 days
70	7×10^{-8} s	4.9×10^{-6} s	$\approx 37,000$ years
90	9×10^{-8} s	8.1×10^{-6} s	≈ 39 billion years
100	10^{-7} s	10^{-5} s	≈ 40 quintillion years

By modest input sizes, exponential-time algorithms already exceed any realistic time horizon.

Reframing the conjecture

Are problems that appear to require exponential time actually solvable in polynomial time?

- Verification can stay easy while search explodes combinatorially.
- That asymmetry is the practical face of the P vs. NP problem.
- Better hardware helps, but exponential growth still wins.

Intuition: Verification vs. Search

A useful mental model:

- **Verifying** a candidate solution may be fast.
- **Finding** that solution may require massive search.

Example: **Sudoku**

- Checking a completed grid is quick.
- Constructing a valid solution can require extensive search.

NP-Complete Problems

A problem is NP-complete if:

- it belongs to NP, and
- every problem in NP reduces to it in polynomial time.

Examples:

- Traveling Salesman Problem
- Subset Sum
- Knapsack

A polynomial-time algorithm for any NP-complete problem would imply

$$P = NP.$$